Université Sultan Moulay Slimane
Faculté des Sciences et Techniques — Béni Mellal
Master AIDC — Intelligence Artificielle et Data Computing

Rapport de Travaux Pratiques

Systèmes Multi-Agents — TP 2

Simulation multi-agent avec Mesa 3 : formation de l'opinion publique

Réalisé par :

Fatima-ezzahra AMARZOUG

Encadré par :

Pr. Lhoucine DAOUDI

Dépôt GitHub :

https://github.com/fatimaezzahra-max/p2-mesa-opinion-publique

Année universitaire : 2026/2027

Table des matières

1 Objectif du TP **2**

2 Description du modèle **2**

3 Implémentation **2**

3.1 L'agent 2

3.2 Le modèle 3

4 Résultats de la simulation **3**

5 Réponses aux questions **4**

5.1 Compréhension théorique 4

5.2 Technique et manipulation de Mesa 3 5

5.3 Ouverture et esprit critique 6

6 Conclusion **7**

1 Objectif du TP

Le but de ce TP est de comprendre comment l'opinion d'une population peut se former à partir d'interactions très simples entre individus. Pour cela, on utilise **Mesa 3**, une bibliothèque Python de modélisation à base d'agents (système multi-agent).

Chaque individu est un agent qui possède une opinion. Il ne connaît que ses voisins directs et ajuste son opinion en fonction d'eux. Il n'y a aucun contrôle central. On observe ensuite ce qui apparaît à l'échelle de toute la population : un consensus, une polarisation ou des groupes d'opinion. C'est ce qu'on appelle l'**émergence**.

Dans ce compte rendu, nous présentons le modèle, notre implémentation, les résultats de la simulation, puis les réponses aux questions du TP. Le code complet est disponible sur le dépôt GitHub indiqué sur la page de garde.

2 Description du modèle

Le modèle suit le protocole ODD donné dans le TP.

- **Entités :** les agents représentent des individus placés sur une grille 10×10 (un agent par case, soit 100 agents). La grille est un tore : les bords sont reliés entre eux.
- **Variable d'état :** l'opinion $o_i \in [0, 1]$ (0 = totalement contre, $0{,}5$ = neutre, 1 = totalement pour). Elle est tirée au hasard au départ.
- **Processus :** à chaque tick, un agent calcule la moyenne $\bar{o}_{\mathcal{N}_i}$ des opinions de ses 8 voisins (voisinage de Moore) et rapproche son opinion de cette moyenne :

$$o_i^{t+1} = o_i^t + \alpha \left(\bar{o}_{\mathcal{N}_i}^t - o_i^t \right), \qquad \alpha \in [0, 1]. \tag{1}$$

- **Paramètre :** α est le niveau d'influence sociale.
- **Observation :** opinion moyenne, variance (dispersion) et trajectoires individuelles.

3 Implémentation

3.1 L'agent

La classe `CitizenAgent` possède une opinion aléatoire. Dans sa méthode `step()`, l'agent récupère ses voisins, calcule leur opinion moyenne et met à jour son opinion avec la formule ci-dessus.

```
class CitizenAgent(mesa.Agent):
    def __init__(self, model, alpha):
        super().__init__(model)
        self.opinion = self.model.random.random()
        self.alpha = alpha

    def step(self):
```

```
        neighbors = self.model.grid.get_neighbors(
            self.pos, moore=True, include_center=False)
        if len(neighbors) > 0:
            mean_opinion = np.mean([n.opinion for n in neighbors])
            self.opinion = self.opinion + self.alpha * (
                mean_opinion - self.opinion)
```

3.2 Le modèle

La classe `OpinionModel` crée la grille (`MultiGrid` avec `torus=True`), place un agent sur chaque case et prépare le `DataCollector` (moyenne et variance pour le modèle, opinion pour chaque agent). À chaque tick, le modèle collecte les données puis active tous les agents dans un ordre aléatoire avec `self.agents.shuffle_do("step")`.

4 Résultats de la simulation

Paramètres utilisés : grille 10×10, $\alpha = 0{,}75$, 50 ticks, graine 42.

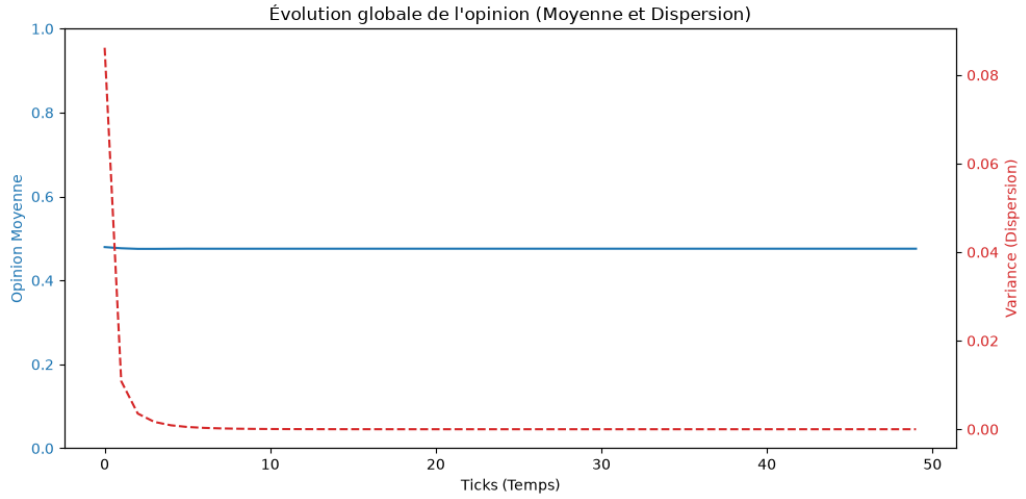


FIGURE 1 – Évolution globale de l'opinion : moyenne (bleu) et variance (rouge, pointillés).

L'opinion moyenne reste presque constante, autour de 0,48 (de 0,4796 au départ à 0,4757 à la fin). La variance, elle, chute très vite : elle vaut environ 0,086 au départ, 0,011 après un tick, 5×10^{-4} au tick 5 et 5×10^{-5} au tick 10, pour arriver à environ 10^{-10} à la fin. Les agents finissent donc par avoir pratiquement la même opinion.

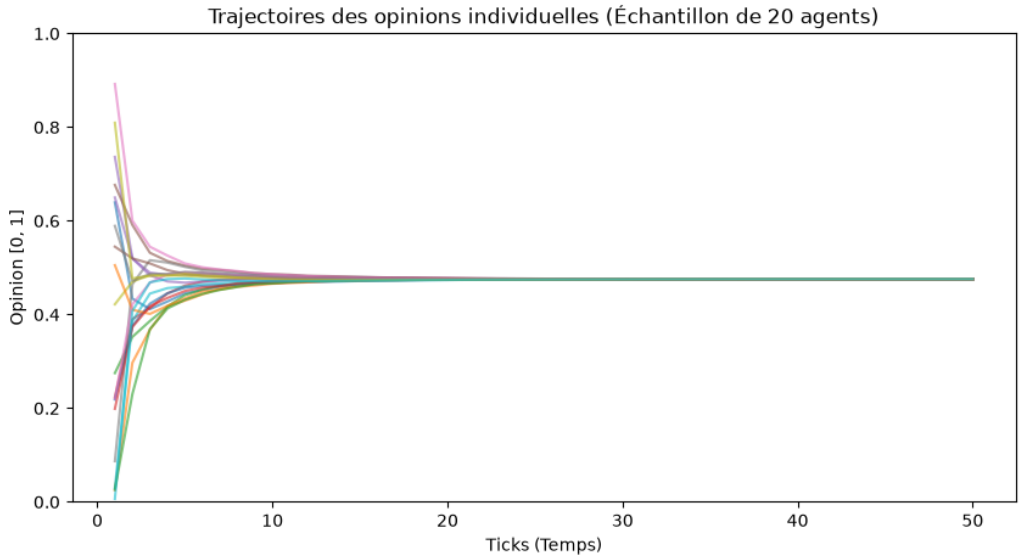


FIGURE 2 – Trajectoires des opinions individuelles (échantillon de 20 agents).

Sur la figure 2, les opinions de départ sont très différentes (de 0 à 0,9), puis toutes les courbes se rejoignent en une dizaine de ticks autour de la même valeur ($\approx 0{,}48$) et restent ensuite stables.

5 Réponses aux questions

5.1 Compréhension théorique

1. Impact du paramètre d'influence α sur la vitesse de convergence globale. Le paramètre α détermine le degré d'influence des voisins sur l'opinion d'un agent. Si α est proche de 0, les opinions changent lentement : les courbes restent presque plates. Si $\alpha = 1$, l'agent adopte directement la moyenne des opinions de ses voisins. Donc, plus α est grand, plus le changement est important à chaque étape et plus la convergence est rapide.

Pour vérifier cela, nous avons relancé la simulation avec plusieurs valeurs de α (moyenne sur 20 graines différentes). Le tableau ci-dessous donne le nombre de ticks nécessaires pour que la variance passe sous 10^{-3}.

α	Ticks pour atteindre une variance $< 10^{-3}$
0,01	non atteint en 300 ticks
0,05	≈ 101
0,25	≈ 19
0,50	≈ 9
1,00	≈ 4

TABLE 1 – Vitesse de convergence selon α.

2. Mécanisme mathématique qui rend la polarisation ou la création de groupes difficile à long terme. La formule rapproche progressivement l'opinion de chaque agent de la moyenne des opinions de ses voisins. Elle peut s'écrire $o_i^{t+1} = (1-\alpha)\, o_i^t + \alpha\, \bar{o}_{\mathcal{N}_i}^t$: c'est une moyenne pondérée de deux opinions qui existent déjà. Comme l'agent se rapproche toujours de cette moyenne, il ne peut jamais s'en éloigner, et les différences entre les opinions diminuent progressivement. De plus, tous les voisins comptent quelle que soit leur opinion, et la grille est connectée (avec le tore), donc aucun groupe ne reste isolé. Le modèle favorise donc un consensus global plutôt que la polarisation ou la formation de groupes séparés.

Il faut quand même nuancer : dans ce modèle le consensus est favorisé, mais il n'est pas garanti dans toutes les configurations possibles (par exemple si l'on change la règle d'influence, comme à la question 7).

3. Manifestation de l'émergence dans les graphiques. L'émergence se manifeste lorsque les interactions entre les agents produisent un comportement collectif sans intervention extérieure. Ici, chaque agent ne connaît que ses 8 voisins et ne sait rien de la population entière. Pourtant, sur la figure 1, la variance diminue jusqu'à presque zéro, et sur la figure 2 les trajectoires individuelles se rapprochent jusqu'à se confondre. Le consensus est donc une propriété du groupe, qui apparaît sans que personne l'ait programmée.

5.2 Technique et manipulation de Mesa 3

4. Intérêt de `self.agents.shuffle_do("step")`. Cette instruction permet d'activer tous les agents dans un ordre aléatoire et d'exécuter leur méthode `step()`. Dans Mesa 3, les agents sont gardés automatiquement dans l'objet `self.agents`, donc on n'a plus besoin de scheduler ni de liste à gérer soi-même. L'ordre aléatoire évite de toujours traiter les agents dans le même ordre : sinon, les premiers verraient toujours des opinions « anciennes » et les derniers des opinions « récentes », ce qui créerait un effet artificiel. Cela permet de représenter des interactions sociales moins dépendantes d'un ordre fixe.

5. Différence entre `model_reporters` et `agent_reporters`. `model_reporters` permet de collecter des informations globales sur la population, comme l'opinion moyenne et la variance (une valeur par tick). `agent_reporters` permet de suivre les informations de chaque agent individuellement (une valeur par agent et par tick). Les deux sont nécessaires pour observer à la fois l'évolution générale des opinions et les changements de chaque individu. Par exemple, la moyenne seule ne suffit pas : deux groupes à 0 et à 1 donneraient aussi une moyenne de 0,5, et c'est la variance qui permet de faire la différence.

6. Effet du paramètre `torus=True`. Le paramètre `torus=True` relie les bords opposés de la grille. Ainsi, un agent situé sur un bord peut avoir des voisins de l'autre côté de la grille. Cela permet de conserver le même principe de voisinage même pour les agents

situés aux extrémités. Par exemple, l'agent en $(0, 0)$ a bien 8 voisins (dont $(9, 9)$, $(0, 9)$ et $(9, 0)$), alors que sans tore il n'en aurait que 3.

5.3 Ouverture et esprit critique

7. Biais de confirmation avec un seuil de tolérance ϵ. Pour introduire le biais de confirmation, nous avons modifié la méthode `step()` de l'agent afin qu'il soit influencé uniquement par les voisins dont l'opinion est suffisamment proche de la sienne, selon un seuil `epsilon`.

```
class BiasedCitizenAgent(CitizenAgent):
    def __init__(self, model, alpha, epsilon):
        super().__init__(model, alpha)
        self.epsilon = epsilon

    def step(self):
        neighbors = self.model.grid.get_neighbors(
            self.pos, moore=True, include_center=False)
        close = [n.opinion for n in neighbors
                 if abs(n.opinion - self.opinion) < self.epsilon]
        if len(close) > 0:
            mean_close = np.mean(close)
            self.opinion = self.opinion + self.alpha * (
                mean_close - self.opinion)
```

Cette modification peut entraîner la formation de groupes d'opinions similaires et favoriser la polarisation, car les agents échangent principalement avec des personnes qui partagent des opinions proches des leurs.

Nous l'avons testé avec $\alpha = 0{,}5$, $\epsilon = 0{,}2$, 100 ticks et la graine 42.

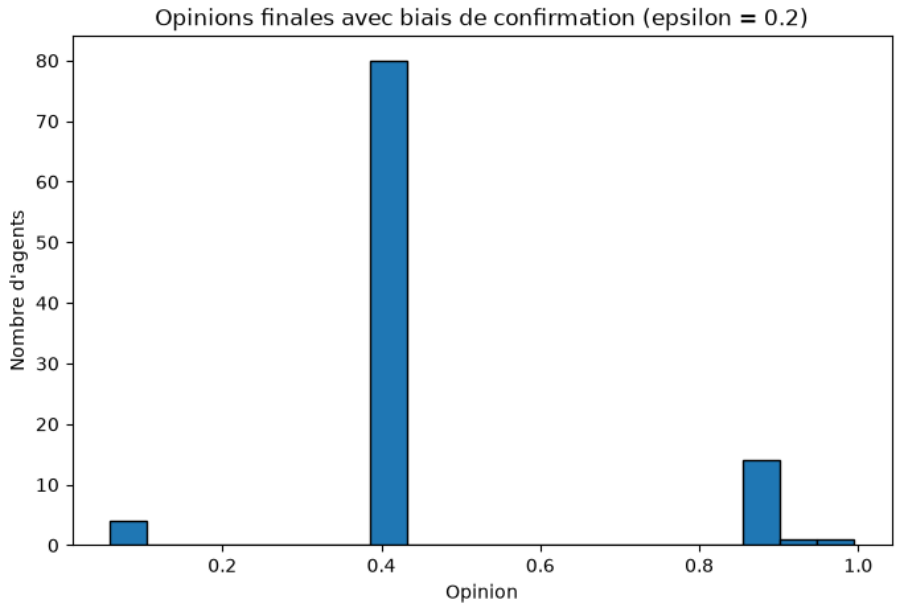


FIGURE 3 – Évolution de la dispersion des opinions avec biais de confirmation.

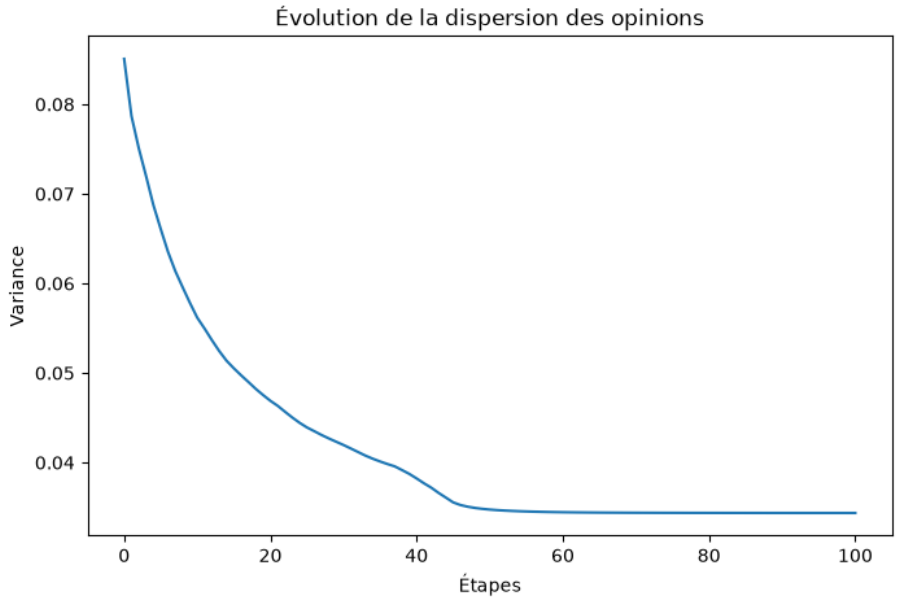


FIGURE 4 – Opinions finales avec biais de confirmation ($\epsilon = 0{,}2$).

La différence avec le modèle de base est nette. La variance (figure 3) baisse de 0,085 à environ 0,034 puis se stabilise vers le tick 60, sans jamais tendre vers zéro : il n'y a donc pas de consensus. L'histogramme (figure 4) montre que les opinions finales se répartissent en plusieurs groupes : environ 80 agents autour de 0,4, un petit groupe de 4 agents vers 0,07 et environ 16 agents vers 0,9. Ce sont des groupes qui ne s'influencent plus entre eux, comme des chambres d'écho.

8. Structure pour modéliser un vrai réseau social. On peut utiliser un graphe avec `mesa.space.NetworkGrid`, basé sur une structure de réseau telle que `networkx.Graph` ou `networkx.DiGraph`.

- Les nœuds représentent les utilisateurs.
- Les arêtes représentent les relations ou les abonnements.
- Les influenceurs sont représentés par des nœuds ayant beaucoup de connexions.
- Les utilisateurs isolés ont peu de connexions, ou aucune.

Un graphe orienté (`DiGraph`) est particulièrement utile si on veut représenter le fait que l'utilisateur A suit B sans que B suive nécessairement A.

6 Conclusion

Ce TP nous a permis de comprendre comment des règles locales très simples produisent un comportement collectif. Avec le modèle de base, les agents convergent vers un consensus, d'autant plus vite que α est grand. En ajoutant un seuil de tolérance ϵ, on obtient à la place plusieurs groupes d'opinion qui restent séparés. Nous avons aussi appris à utiliser les outils de Mesa 3 (`AgentSet`, `shuffle_do`, `DataCollector`, `MultiGrid`).

Code source : https://github.com/fatimaezzahra-max/p2-mesa-opinion-publique